POLITECNICO DI TORINO · Master Degree in Data Science

Official Examination Session 3

Date: July 7, 2026 | **Time Allotted:** 1 Hour 30 Minutes | **Total Points:** 16.00 Points

Session Scope
Summer Examination Session

§1 Exam Session 3 — July 7, 2026

Session Overview: This examination comprises 7 questions covering autonomous driving multi-class perception architecture, mechanistic interpretability (features, circuits, universality, polysemanticity), LIME interpretable image representations, SHAP vs. LIME comparative analysis, Partial Dependence Plots (mathematics, computation, limitations), TCAV algorithmic mechanics and algorithmic bias detection, and gradient noise origins (total: 16.00 points).

Question 1: Autonomous Vehicle Camera Perception: Architecture, XAI Methods, and Recommendation

Open Inquiry **5.00 Points**

You want to design an AI system for an autonomous vehicle that analyzes camera images to recognize traffic signs, pedestrians, lanes, and obstacles. The system needs to produce a prediction, such as "stop sign," "pedestrian crossing," or "obstacle ahead," with an associated confidence score.

How would you design the system so that it provides understandable explanations to engineers, safety auditors, and regulators? Discuss at least three explainability methods that could be used, specifying whether they are by-design or post-hoc, local or global, and the type of explanation representation they provide (e.g., local rules, feature importance). Among the discussed methods, choose the one you would implement in the final system and justify your choice.

✓ Executive Solution Summary

Autonomous vehicle vision systems operate under strict automotive safety standards (ISO 26262, ISO 21448 / SOTIF). Because raw images are non-interpretable tensors $\mathbb{R}^{H\times W\times C}$, explainability methods **must utilize interpretable representations** such as bounding-box proposals, superpixels, or semantic concepts (e.g., shape, color, lettering).

We analyze four distinct methods: (1) **Concept Bottleneck Models (CBMs)** (By-design, Local/Global, Concept attributions), (2) **Prototype Part Networks (ProtoPNet)** (By-design, Local/Global, Case-based visual prototypes), (3) **Generative Counterfactual Explanations for Images** (Post-hoc, Local, Counterfactual synthetic scenes via diffusion/GANs), and (4) **Grad-CAM** (Post-hoc, Local, Coarse spatial saliency).

We explicitly **recommend Concept Bottleneck Models (CBMs)**. CBMs provide auditable symbolic reasoning directly aligned with traffic regulations (e.g., Vienna Convention on Road Signs), eliminate post-hoc approximation unfaithfulness, and crucially empower safety engineers to perform **test-time concept interventions** (e.g., correcting a misdetected attribute under severe weather conditions).

✍ **Comprehensive Technical Walkthrough**

1. Automotive Safety Context & The Problem of Image Representation

Perception systems in autonomous driving are safety-critical. Unexplained misclassifications (e.g., confusing an occluded STOP sign for a speed sign) lead to fatal accidents.

The Imperative of Interpretable Representations for Vision Data

A camera feed delivers high-dimensional raw tensors $x \in \mathbb{R}^{H \times W \times 3}$. **Individual raw pixels possess zero semantic interpretability to safety regulators or perception engineers.** Therefore, any explainability pipeline must bridge the representation gap via:

- **Object Proposals / Bounding Boxes:** Regions extracted by an upstream detector (e.g., YOLO / Faster R-CNN) isolating candidate objects.
- **High-Level Semantic Concepts:** Human-defined visual attributes $c \in \mathcal{C}$ (e.g., *"octagonal contour"*, *"red background"*, *"white lettering 'STOP"'*, *"upright bipedal human silhouette"*, *"yellow reflective lane striping"*).

2. Detailed Discussion of Candidate Explainability Methods

Method 1: Concept Bottleneck Models (CBMs) [By-Design | Local & Global]

- **Architecture:** Maps input images through a two-stage sequential architecture:

$$x \xrightarrow{g} \hat{\mathbf{c}} \xrightarrow{f} \hat{y} \tag{1}$$

  where $g(x)$ is a deep convolutional/vision transformer backbone predicting K symbolic concepts $\hat{\mathbf{c}} \in [0,1]^K$, and $f(\hat{\mathbf{c}})$ is an inherently interpretable linear or rule-based classifier producing the target class and confidence score.

- **Explanation Representation:** *Concept Attribution Vectors*: The model explains $\hat{y} =$ "Stop Sign" by showing the active concepts ($\hat{c}_{\text{octagonal}} = 0.98$, $\hat{c}_{\text{red_color}} = 0.95$, $\hat{c}_{\text{text_STOP}} = 0.91$) and their linear contribution weights.
- **Crucial Requirement on Annotated Data:** CBMs require training data annotated with both class labels and concept attributes. In autonomous driving, this is readily feasible because standard industry benchmarks (e.g., BDD100K, Cityscapes, nuScenes) already provide detailed segmentations, pedestrian attribute tags, and traffic sign metadata. Alternatively, zero-shot Vision-Language Models (e.g., CLIP) can automatically pseudo-label concepts across the training set.
- **Safety Intervention:** If heavy snowfall occludes the letters "STOP", a perception engineer or secondary radar/LiDAR fusion subsystem can intervene directly on the concept bottleneck: $\hat{c}_{\text{octagonal}} = 1.0 \wedge \hat{c}_{\text{red}} = 1.0 \implies \hat{y} =$ "Stop Sign".

Method 2: Prototype Part Networks (ProtoPNet) [By-Design | Local & Global]

- **Architecture:** Compares latent convolutional feature patches against learned visual prototype vectors $\{p_j\}_{j=1}^{M}$ stored in training memory.
- **Explanation Representation:** *Visual Example-Based*: The system justifies the prediction "Pedestrian Crossing" by displaying: *"This image patch has high similarity (0.93) to Training Prototype 12 (standard zebra crossing stripes) and Prototype 19 (walking human legs)."*

- **Pros & Cons:** Explains visually without concept labels; however, prototypes can occasionally latch onto background pavement artifacts.

Method 3: Counterfactual Explanations for Image Data [Post-Hoc | Local]

- **Architecture & Adaptation for Image Data:** In tabular data, counterfactuals perturb individual features. In image data, **pixel-level perturbation is invalid because it produces off-manifold adversarial noise**. To generate valid visual counterfactuals, one must employ a **deep generative model** (e.g., Latent Diffusion Models, StarGAN, or CycleGAN):

$$x_{\text{CF}} = G\left(E(x) + \delta_z\right) \quad \text{s.t.} \quad f(x_{\text{CF}}) = y_{\text{alternative}}, \quad \|\delta_z\| \text{ minimized} \tag{2}$$

  where $E(x)$ maps the image to a generative latent space and G reconstructs a realistic counterfactual scene.

- **Explanation Representation:** *Visual Image Comparison*: Displays a realistic synthetic image showing: *"If the red hexagonal sign had circular geometry, the prediction would change to 'Speed Limit'."*

- **Limitation:** Extremely high inference latency (seconds per image), making real-time onboard explanation impossible; risk of generative hallucinations.

Method 4: Grad-CAM [Post-Hoc | Local]

- **Mechanism:** Computes gradients of the target class score with respect to convolutional feature maps: $L^c_{\text{Grad-CAM}} = \text{ReLU}\left(\sum_k \alpha^c_k A^k\right)$.

- **Explanation Representation:** *Visual Heatmap Overlay*: Coarse spatial highlighting indicating where in the camera frame the network attended.

- **Limitation:** Lacks semantic precision; cannot convey whether an attended pedestrian was detected due to pose, motion, or reflective clothing.

3. Comparative Synthesis Across Stakeholder Dimensions

Method	Paradigm & Scope	Representation	Data Requirements	Primary Stakeholder Value
CBM	By-Design · Local/Global	Concept attributions	Concept-annotated images (BDD100K/CLIP)	**Regulators & Engineers:** Traceable rules & runtime intervention
ProtoPNet	By-Design · Local/Global	Visual prototypes	Class labels only	**Engineers:** Case-based visual validation
Counterfactual	Post-Hoc · Local	Synthetic image	Pre-trained generative model (Diffusion)	**Safety Auditors:** Offline edge-case exploration
Grad-CAM	Post-Hoc · Local	Spatial heatmap	Standard supervised data	**Engineers:** Quick sanity check of spatial gaze

4. Final Recommendation and Rigorous Justification

Recommendation: We formally recommend designing the system around an explainable-by-design **Concept Bottleneck Model (CBM)**, integrated with spatial object detection proposals.

Justification Based on Stakeholder Requirements:

J1. For Regulators (Traceability & Legal Certification): Traffic regulations (such as the Vienna Convention on Road Signs and Signals) explicitly codify signs through discrete symbolic rules: an octagonal shape combined with a red surface and white text is legally a STOP sign. CBMs mirror this legal codification natively in the bottleneck layer, providing unambiguous compliance audits that satisfy UNECE WP.29 automotive AI regulations.

J2. For Safety Auditors (Architectural Faithfulness & No Approximation Error): Post-hoc methods (such as LIME or Grad-CAM) suffer from approximation fidelity gaps and visual artifacts that can deceive safety auditors. CBMs are *faithful by design*: the prediction is mathematically bounded to be a deterministic function of the predicted concepts: $\hat{y} = f(\hat{\mathbf{c}})$.

J3. For Perception Engineers (Diagnostics & Runtime Intervention): When an edge case occurs (e.g., an autonomous vehicle fails to detect a pedestrian in torrential rain), engineers can dissect the exact point of failure: did the system fail to detect the concept *"upright human posture"* (a perception failure in $g(x)$), or did the classifier fail to trigger the brake (a logic failure in $f(c)$)? Crucially, CBMs support **runtime concept intervention** by sensor fusion fallbacks, directly enforcing safety.

★ Examiner Feedback & Common Pitfalls (Prof. Eliana Pastor)

In official grading, students lost points (−0.50 points total) on two specific omissions:

1. **Omitting the need for annotated data in Concept-Based models (−0.25 points):** Candidates must address how concept annotations are obtained (e.g., leveraging existing annotated driving datasets like BDD100k or using zero-shot VLM pseudo-labeling).
2. **Failing to discuss how counterfactuals are adapted to image data (−0.25 points):** Simply saying "perturb the features" is incorrect for images. Candidates must explain that generating visual counterfactuals requires deep generative models (Diffusion Models/GANs) to alter latent factors rather than raw pixels.

➥ Course Theory Mapping (Notes & Slides)

NOTES/XAI_Theory.pdf – Chapter 7: "Concept-based Explainable AI: CBMs" (pp. 98–106), Chapter 9: "Counterfactual Explanations for Image Data" (pp. 134–138), and Chapter 5: "Grad-CAM" (pp. 72–76).

¶ Alternative Perspectives & Methodological Variants

Object-Centric Slot Attention Models (Locatello et al.) extract unsupervised object slots corresponding to individual actors (pedestrians, vehicles, obstacles) directly from scene dynamics, providing by-design modularity without requiring manual pixel-level semantic masks.

Question 2: Mechanistic Interpretability: Features, Circuits, Universality, and Polysemanticity Open Inquiry 2.50 Points

Describe the goal of mechanistic interpretability in neural networks. In your answer, explain the notions of features, circuits, and universality. Then, discuss one challenge that makes mechanistic interpretability difficult, especially in large neural networks or LLMs.

✓ Executive Solution Summary

The primary goal of **mechanistic interpretability** is to reverse-engineer neural networks from raw numeric weights and activations into human-understandable algorithmic programs and computational subroutines. Rather than treating representations as statistical correlations, it seeks to uncover the exact causal algorithms computing network outputs.
It is grounded in three foundational notions: (1) **Features** (the atomic scalar variables represented by directions in activation space), (2) **Circuits** (computational subgraphs formed by features and connecting weights that execute algorithms), and (3) **Universality** (the emergence of identical features and circuits across different models trained on the same task).
A central challenge is **polysemanticity** driven by **superposition**: models represent vastly more features than they have physical dimensions ($M \gg D$) by packing them almost orthogonally, causing single neurons to activate for multiple unrelated semantic concepts.

✍ Comprehensive Technical Walkthrough

1. The Primary Goal of Mechanistic Interpretability

Traditional XAI methods (e.g., LIME, SHAP, Saliency Maps) operate via *attribution*—identifying which inputs were statistically correlated with an output. In contrast, **mechanistic interpretability** treats neural networks as compiled computational graphs. Its goal is to **reverse-engineer the network into human-readable source code**, identifying:

$$\text{Raw Weights } \theta \xrightarrow{\text{Reverse-Engineering}} \text{Algorithms, Circuits, and Discrete Logic} \tag{3}$$

2. The Three Speculative Claims (Olah et al.)

A. Features (Atomic Units of Representation)

- **Mechanistic Definition:** A *feature* is a scalar property of the input that the model represents internally. Formally, a feature corresponds to a specific **direction vector** $\mathbf{v} \in \mathbb{R}^d$ in the activation space of a layer. The magnitude of the projection $\langle \mathbf{h}(x), \mathbf{v} \rangle$ indicates the presence or intensity of that property.
- **Examples:** In CNNs, features range from Gabor-like oriented edges to curve detectors and car-body detectors. In LLMs, features represent syntax markers (e.g., past tense), grammatical roles, or factual associations (e.g., "capital of France").

B. Circuits (Algorithmic Subgraphs)

- **Mechanistic Definition:** A *circuit* is a computational subgraph of the neural network consisting of a set of features (nodes) interconnected by model weights (edges) that together execute an end-to-end algorithmic operation:

$$\mathcal{C} = (\mathcal{V}_{\text{features}}, \mathcal{E}_{\text{weights}}) \tag{4}$$

- **Empirical Examples:**
  - *Induction Heads in Transformers:* A two-head circuit across consecutive attention layers that implements pattern completion: if token sequence $[A][B]$ appeared earlier in context, and token $[A]$ appears again, the circuit attends back and copies $[B]$:

$$\dots [A][B] \dots [A] \xrightarrow{\text{Induction Circuit}} [B] \tag{5}$$

  - *Indirect Object Identification (IOI) Circuit:* A 26-head circuit in GPT-2 Small that parses sentences like *"When Mary and John went to the store, John gave a drink to ..."* and routes attention to predict *"Mary"*.

C. Universality

- **Mechanistic Definition:** The empirical finding that across different models, training runs, initializations, and architectures, **the same features and circuits independently emerge** when optimizing for the same task.
- **Significance:** Implies that circuits are not random training artifacts, but represent universal mathematical algorithms inherent to the problem distribution itself.

3. Major Open Challenge: Polysemanticity and the Superposition Hypothesis

The Phenomenon of Polysemanticity

In an ideal interpretable network, individual neurons would be *monosemantic* (each neuron activating for exactly one clean concept). In practice, deep networks exhibit severe **polysemanticity**: **a single individual neuron fires for completely unrelated, disjoint concepts** (e.g., an individual neuron activating simultaneously for academic citations, front legs of cats, and German adjectives).

The Superposition Hypothesis

Anthropic researchers (Elhage et al., 2022) proved that polysemanticity is an optimal response to dimensional constraints:

- A model needs to represent M distinct real-world concepts, but its hidden layer has only D dimensions, where $M \gg D$.
- By exploiting the geometry of high-dimensional vector spaces (where an exponential number of vectors can be *almost orthogonal*), the network packs M non-orthogonal features into D dimensions using **superposition**:

$$\mathbf{h}(x) = \sum_{i=1}^{M} c_i(x)\mathbf{v}_i \quad \text{with} \quad \langle \mathbf{v}_i, \mathbf{v}_j \rangle \approx 0 \quad (i \neq j) \tag{6}$$

- Because features are non-orthogonal, individual coordinate axes (standard neurons) project onto multiple features simultaneously, rendering neuron-level inspection uninterpretable.

Frontier Solution: Sparse Autoencoders (SAEs)

To resolve polysemanticity, modern mechanistic interpretability trains overcomplete **Sparse Autoencoders** on the residual stream activations $\mathbf{h} \in \mathbb{R}^D$:

$$\mathbf{z} = \text{ReLU}\left(W_{\text{enc}}\mathbf{h} + \mathbf{b}_{\text{enc}}\right), \quad \hat{\mathbf{h}} = W_{\text{dec}}\mathbf{z} + \mathbf{b}_{\text{dec}} \quad \text{s.t.} \quad \mathcal{L} = \|\mathbf{h} - \hat{\mathbf{h}}\|_2^2 + \lambda\|\mathbf{z}\|_1 \tag{7}$$

with hidden dimension $F \gg D$ ($F \approx 32D$). The L_1 sparsity penalty forces the autoencoder to untangle superposed features into monosemantic latent directions.

★ Examiner Feedback & Common Pitfalls (Prof. Eliana Pastor)

Prof. Eliana Pastor penalized candidates (deducting −0.50 points) who gave superficial descriptions of features and circuits. A top-scoring response must define features mathematically (directions in activation space), circuits as algorithmic subgraphs of weights and features with concrete examples (induction heads / IOI), and explain how the **superposition hypothesis** geometrically causes polysemanticity.

➥ Course Theory Mapping (Notes & Slides)

NOTES/XAI_Theory.pdf – Chapter 11: "Mechanistic Interpretability", Sections: "The Three Speculative Claims", "Circuits", and "Mechanistic Interpretability in LLMs: Polysemanticity and Sparse Dictionary Learning" (pp. 155–168).

¶ Alternative Perspectives & Methodological Variants

Activation Patching (Causal Tracing) (Meng et al.) quantitatively maps circuits by corrupting clean input activations with noise and systematically restoring individual activations to measure their causal effect on the correct logit.

Question 3: LIME Interpretable Representation in Image Classification

MCQ | 1.00 Point

In image classification, what is a typical interpretable representation used by LIME?

a. Gradients of the loss function

b. Image patches or superpixels

c. Neural network weights

d. Individual pixels only

✓ Executive Solution Summary

The correct answer is **b**. In image classification, LIME groups contiguous, perceptually homogeneous pixels into **image patches or superpixels** (via segmentation algorithms like SLIC or Quickshift). The interpretable representation is a binary vector $z' \in \{0,1\}^M$ indicating whether each superpixel patch is visible (1) or occluded/masked (0).

➤ Systematic Analysis of Multiple Choice Options

(b) [CORRECT] Image patches or superpixels

LIME operates on an interpretable domain $z' \in \{0,1\}^M$ where each coordinate represents an image patch or superpixel. The perturbation function $h_x(z')$ restores superpixels set to 1 and replaces superpixels set to 0 with an average color, grey mask, or blurred patch.

(a) [INCORRECT] Gradients of the loss function

Gradients are continuous mathematical derivatives used in white-box backpropagation methods (Vanilla Gradient, Integrated Gradients), whereas LIME is a black-box perturbation technique.

(c) [INCORRECT] Neural network weights

Weights are the internal parameters of the black-box model, not an input representation presented to the user.

(d) [INCORRECT] Individual pixels only

Raw pixels are completely uninterpretable in isolation to humans and computationally intractable: an image contains 10^5–10^6 pixels, making local surrogate fitting impossible without grouping them into superpixels.

✍ **Comprehensive Technical Walkthrough**

LIME relies on the principle of **Interpretable Data Representations**. While the original model operates on $x \in \mathbb{R}^d$ (e.g., $d = 224 \times 224 \times 3 \approx 150,528$ pixel dimensions), humans reason about visual objects.



The mapping function $h_x : \{0,1\}^M \to \mathbb{R}^{H \times W \times 3}$ transforms a binary perturbation vector z' back into pixel space:

$$[h_x(z')]_p = \begin{cases} x_p & \text{if } z'_j = 1 \quad (p \in \text{Superpixel}_j) \\ \mu_{\text{background}} & \text{if } z'_j = 0 \quad (p \in \text{Superpixel}_j) \end{cases} \tag{8}$$

Fitting a sparse linear surrogate on z' yields feature attribution coefficients directly tied to visual superpixels.

★ **Examiner Feedback & Common Pitfalls (Prof. Eliana Pastor)**

Never answer "individual pixels". A primary conceptual contribution of Ribeiro et al. was recognizing that individual pixels carry zero human interpretability and that segmentation into superpixels is mandatory for image explanations.

➥ **Course Theory Mapping (Notes & Slides)**

NOTES/XAI_Theory.pdf – Chapter 5: "Local Surrogate Models: LIME", Section: "The Three Pillars of LIME: Interpretable Data Representations" (pp. 42–44).

¶ **Alternative Perspectives & Methodological Variants**

Occlusion sensitivity (Zeiler & Fergus) uses sliding square bounding boxes across the image rather than superpixel boundaries, but superpixels provide superior boundary alignment with natural object contours.

Question 4: Relation Between SHAP and Shapley Values & Comprehensive Comparison with LIME — Open Inquiry — 2.50 Points

Explain the relation between SHAP and Shapley values from cooperative game theory. Motivate why this relation is useful for explainability.
Then, compare SHAP with LIME, highlighting common points and differences between the two methods.

✓ Executive Solution Summary

SHAP formulates local model explainability through the lens of cooperative game theory: the input features act as *players* in a coalition, and the model prediction serves as the *game payout*. SHAP computes the **Shapley values**—the unique allocation that distributes the payout fairly among players by averaging marginal contributions across all possible coalitions. This relation provides a solid **axiomatic guarantee** (Efficiency, Symmetry, Dummy Player, Additivity), ensuring mathematically fair attribution.

SHAP and LIME share foundational principles: both are **strictly local**, **post-hoc**, and **model-agnostic** methods that train a **weighted linear surrogate model** over perturbed **interpretable representations** ($z' \in \{0,1\}^M$).

They differ fundamentally in: (1) *Theoretical Foundation* (SHAP uses mathematically constrained Shapley kernel weights yielding game-theoretic axioms; LIME uses an empirical heuristic distance kernel), (2) *Axiomatic Guarantees* (SHAP guarantees 4 axioms; LIME provides none), (3) *Computational Complexity* (LIME is fast with few samples; SHAP requires exponential coalition evaluation or approximations), (4) *Stability* (LIME has high stochastic sampling variance; SHAP is significantly more consistent), and (5) *Feature Interactions* (SHAP evaluates coalitions; LIME assumes local linear independence).

✍ **Comprehensive Technical Walkthrough**

1. SHAP and Shapley Values: Formal Relation and Game-Theoretic Mapping

Lundberg & Lee (2017) unified local surrogate methods under cooperative game theory:

- **Grand Coalition ($N = \{1, 2, \ldots, M\}$):** The set of all available input features.
- **Coalition Subset ($S \subseteq N$):** A subset of present features.
- **Characteristic Payout Function $v(S)$:** The expected prediction when only features in S are known, integrating over absent features:

$$v(S) = \mathbb{E}_{X_{\bar{S}}}\left[f(x_S, X_{\bar{S}})\right] \tag{9}$$

- **Shapley Value Formula:**

$$\phi_i(v) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!\,(|N| - |S| - 1)!}{|N|!}\left[v(S \cup \{i\}) - v(S)\right] \tag{10}$$

Why This Relation Is Invaluable for Explainability

Shapley values provide the **only mathematically proven solution** that simultaneously satisfies four essential axioms of fair credit allocation:

a. **Efficiency:** $\sum_{i=1}^{M} \phi_i = f(x) - \mathbb{E}[f(X)]$. The explanation accounts for 100% of the prediction shift.

b. **Symmetry:** Identical features receive identical attributions: $v(S \cup \{i\}) = v(S \cup \{j\}) \implies \phi_i = \phi_j$.

c. **Dummy Player:** Non-informative features receive strictly zero attribution: $v(S \cup \{i\}) = v(S) \implies \phi_i = 0$.

d. **Additivity / Monotonicity:** Prevents consistency paradoxes where a feature's true contribution increases but its reported attribution drops.

2. Comprehensive Comparison: SHAP versus LIME

A. Common Points (Shared Methodological DNA)

C1. Strictly Local Scope: Both methods are local explainers. They explain the decision for a specific query point x, not the global behavior across the dataset. (The "L" in LIME explicitly stands for *Local*).

C2. Post-Hoc & Model-Agnostic: Both treat the machine learning model f as a black box, requiring only the ability to pass inputs and read output probabilities.

C3. Binary Interpretable Representations: Both map complex inputs x into a simplified binary vector $z' \in \{0,1\}^M$ representing presence (1) or absence/occlusion (0) of features, tokens, or superpixels.

C4. Local Perturbation Mechanism: Both generate synthetic perturbations in the neighborhood of x by flipping bits in z', mapping them back to feature space $h_x(z')$, and evaluating $f(h_x(z'))$.

C5. Weighted Linear Surrogate Optimization: Both fit an additive linear explanation model $g(z') = \phi_0 + \sum_{i=1}^{M} \phi_i z'_i$ by solving a weighted least-squares problem:

$$\arg\min_{\phi} \sum_{z'} \pi_x(z') \left(f(h_x(z')) - g(z')\right)^2 + \Omega(g) \tag{11}$$

B. Fundamental Differences

Dimension	LIME (Local Interpretable Model-agnostic Explanations)	SHAP (SHapley Additive exPlanations)
Theoretical Foundation	Heuristic optimization: minimizes local surrogate error with empirical exponential distance kernel	Cooperative Game Theory: uniquely solves for Shapley values via the theoretical Shapley Kernel
Kernel Weighting Function	Heuristic exponential kernel: $\pi_x(z') = \exp\left(-\frac{D(x,z')^2}{\sigma^2}\right)$	Analytically derived Shapley Kernel: $\pi_x(z') = \frac{M-1}{\binom{M}{\lvert z'\rvert}\lvert z'\rvert(M-\lvert z'\rvert)}$
Axiomatic Guarantees	None. Can violate efficiency (attributions do not sum to prediction) and monotonicity	Provably satisfies all 4 game-theoretic axioms (Efficiency, Symmetry, Dummy, Additivity)
Feature Interactions	Assumes local linear independence among perturbed features	Evaluates marginal contributions across all 2^M coalition combinations
Computational Cost	Low to moderate: typically converges well with $N \approx 1,000$ perturbations	Extremely high for KernelSHAP (2^M coalitions); mitigated by TreeSHAP ($\mathcal{O}(TLD^2)$) for trees
Stability & Sensitivity	High variance: different random seeds or kernel widths σ produce conflicting explanations	Deterministic and consistent: higher stability under sampling; exact under TreeSHAP

★ **Examiner Feedback & Common Pitfalls (Prof. Eliana Pastor)**

THIS IS THE NUMBER ONE ERROR IN PAST EXAMS! In previous sessions,

students lost 1.00 full point (scoring 1.50 / 2.50) for claiming that *"LIME is a global surrogate method while SHAP is local."*
LIME is strictly LOCAL. The "L" in LIME stands for *Local*. Both SHAP and LIME are local explanation methods. Failing to state that both are local, omitting the discussion of perturbations and interpretable representations, or failing to detail the computational cost and sampling instability differences will result in heavy deductions.

➥ **Course Theory Mapping (Notes & Slides)**
NOTES/XAI_Theory.pdf – Chapter 5: "Post-Modeling Explainability", Sections: "Local Surrogate Models: LIME" (pp. 42–45), "SHAP" (pp. 54–58), and "Unification of Removal-Based Explanations" (pp. 62–64).

¶ **Alternative Perspectives & Methodological Variants**
Lundberg & Lee proved that **KernelSHAP** is mathematically identical to LIME when LIME's loss function, square penalty, and kernel weighting $\pi_x(z')$ are uniquely replaced by the Shapley Kernel, proving that LIME is a heuristic relaxation of SHAP.

Question 5: Partial Dependence Plot (PDP): Definition, Computation, and Limitations

Open Inquiry **1.50 Points**

Define a Partial Dependence Plot (PDP) in the context of Explainable AI. Explain what type of explanation it provides, how it is computed, and discuss at least one important limitation.

✓ **Executive Solution Summary**
A **Partial Dependence Plot (PDP)** is a **post-hoc, global, model-agnostic** visualization technique that illustrates the marginal effect of one or two features on the predicted outcome of a machine learning model, integrating out the effects of all other features. It provides a **visual feature-effect explanation** (1D curves or 2D heatmaps).
It is computed by evaluating the model on synthetic datasets where the target feature is fixed across a grid while all other features retain their empirical values from the dataset, and averaging the predictions via Monte Carlo integration. Key limitations include the **feature independence assumption** (creating impossible off-manifold samples) and **masking heterogeneous opposing local effects**.

✍ **Comprehensive Technical Walkthrough**

1. Formal Definition and Explanation Type

- **Taxonomy:** Post-hoc, Global, Model-Agnostic.
- **Type of Explanation: Visualization-based, global feature-effect explanation**. It plots a continuous curve (1D) or contour surface (2D) demonstrating whether the model's reliance on a feature is linear, monotonic, step-like, or highly non-linear.
- **Mathematical Definition:** Let $x = (x_S, x_C)$ be the feature vector, where $S \subset \{1, \ldots, p\}$ is the target feature subset ($|S| \in \{1, 2\}$) and $C = \{1, \ldots, p\} \setminus S$ is the complement set containing all remaining features. The partial dependence function is the marginal

expectation:

$$f_S(x_S) = \mathbb{E}_{X_C}\left[f(x_S, X_C)\right] = \int f(x_S, x_C)\, p(x_C)\, dx_C \tag{12}$$

2. Step-by-Step Practical Computation Algorithm

Because the true joint probability distribution $p(x_C)$ is unknown, PDP calculates a **Monte Carlo empirical average** across the dataset $\mathcal{D} = \{x^{(1)}, x^{(2)}, \ldots, x^{(N)}\}$:

Step 1. Define Grid: Select target feature x_S and construct an evenly spaced grid of K evaluation points spanning its empirical range: $\{v_1, v_2, \ldots, v_K\}$.

Step 2. Synthetic Dataset Generation & Black-Box Querying: For each grid point v_k ($k \in \{1, \ldots, K\}$):

a. For every sample $i \in \{1, \ldots, N\}$ in the dataset, replace its feature value $x_S^{(i)}$ with the fixed grid value v_k, leaving all other features $x_C^{(i)}$ unchanged:

$$x_{\text{synth}}^{(i)} = \left(v_k, x_C^{(i)}\right) \tag{13}$$

b. Pass all N synthetic instances through the black-box model f to obtain predictions: $f\left(v_k, x_C^{(i)}\right)$.

c. Compute the arithmetic mean of all predictions:

$$\hat{f}_S(v_k) = \frac{1}{N}\sum_{i=1}^{N} f\left(v_k, x_C^{(i)}\right) \tag{14}$$

Step 3. Plotting: Plot the pairs $\left(v_k, \hat{f}_S(v_k)\right)$ to form the PDP curve.

3. Critical Limitations

Lim 1. The Feature Independence Assumption & Off-Manifold Extrapolation: PDP assumes that X_S and X_C are statistically independent: $p(x_C \mid x_S) = p(x_C)$. In reality, features are almost always correlated (e.g., *height* and *weight*, or *engine displacement* and *fuel consumption*). Fixing $x_S = v_k$ across all samples pairs v_k with impossible values of x_C, creating unfeasible synthetic instances (e.g., a person 2.10m tall weighing 35kg). The model evaluates points in empty regions of feature space where it was never trained, injecting extreme extrapolation errors into the average.

Lim 2. Masking Heterogeneous Opposing Effects: Because PDP plots the *global population average*, it completely conceals heterogeneous local subgroups. If a drug dose increases survival for female patients by $+40\%$ but decreases survival for male patients by -40%, the PDP curve will be a completely flat line, falsely indicating that the feature has zero effect! (Resolved by Individual Conditional Expectation – ICE plots).

Lim 3. Low Dimensionality Constraint: PDP is visually restricted to 1D or at most 2D interaction heatmaps due to human cognitive limits. Higher-order interactions cannot be visualized.

★ Examiner Feedback & Common Pitfalls (Prof. Eliana Pastor)

In official grading, students were penalized (-0.50 points) for **omitting the step-by-step**

computation algorithm. A full-credit answer must explicitly explain: (1) creating the grid of values v_k, (2) replacing feature values across all dataset rows, (3) querying the model, and (4) averaging predictions via Monte Carlo.

➥ **Course Theory Mapping (Notes & Slides)**

NOTES/XAI_Theory.pdf – Chapter 5: "Post-Modeling Explainability", Section: "Partial Dependence Plots (PDPs): Mathematical Definition, Practical Computation, and Limitations" (pp. 37–40).

¶ **Alternative Perspectives & Methodological Variants**

Accumulated Local Effects (ALE) Plots (Apley et al.) overcome the independence limitation by calculating differences conditionally within small local windows $p(x_C \mid x_S)$, avoiding off-manifold extrapolation completely.

Question 6: Testing with Concept Activation Vectors (TCAV) for Bias Detection

Open Inquiry | 2.50 Points

Describe how the Testing with Concept Activation Vectors (TCAV) technique works in a classification scenario where you are required to explain the class **Doctor** and you have a set of images annotated with **Man** and **Woman** (mathematical details are not required).
How can TCAV be employed to reveal model bias?

✓ **Executive Solution Summary**

TCAV is a post-hoc, concept-based interpretability method that quantifies how much a human-understandable concept contributes to a network's class prediction. In this scenario, we learn Concept Activation Vectors (CAVs) for *Man* and *Woman* by training linear classifiers in an intermediate layer's activation space to separate concept images from random negative images. We then compute the directional derivative of the *Doctor* logit along each CAV. The **TCAV score** represents the fraction of *Doctor* inputs positively influenced by that concept.
TCAV reveals **algorithmic model bias** by comparing $\text{TCAV}_{\text{Man, Doctor}}$ against $\text{TCAV}_{\text{Woman, Doctor}}$. If the model yields a high, statistically significant score for *Man* but a near-zero or negative score for *Woman*, it proves that the model relies on societal gender stereotypes rather than clinical diagnostic features (e.g., stethoscopes, white coats).

✍ **Comprehensive Technical Walkthrough**

1. The 5 Core Procedural Steps of TCAV

Let f be the pre-trained black-box neural network, k = "Doctor" the target class, and l a chosen intermediate bottleneck layer with activations $a_l(x) \in \mathbb{R}^{d_l}$.

Step 1. Assemble Concept Datasets (Positive and Negative Sets):

- Collect positive image datasets representing the concepts of interest: P_{Man} and P_{Woman}.
- **Assemble Random Negative Datasets (N_{random}):** Collect multiple sets of

random natural images representing non-concept inputs. *This negative baseline is mathematically essential to establish a reference decision boundary.*

Step 2. Learn Concept Activation Vectors (CAVs):

- Pass P_{Man} and N_{random} through the network to extract intermediate activations $\{a_l(x)\}$.
- Train a binary linear classifier (e.g., Linear SVM) in activation space $\mathbb{R}^{d_l}$ to separate concept activations from random activations.
- The vector orthogonal to the separating hyperplane, pointing in the direction of concept activations, is the **Concept Activation Vector**:

$$\mathbf{v}^l_{\text{Man}} \in \mathbb{R}^{d_l}, \quad \|\mathbf{v}^l_{\text{Man}}\|_2 = 1 \tag{15}$$

Repeat the process with P_{Woman} to obtain $\mathbf{v}^l_{\text{Woman}}$.

Step 3. Compute Directional Derivatives for Class "Doctor":

- Take a set of test images X_{Doctor} correctly classified as doctors.
- For each image $x \in X_{\text{Doctor}}$, compute the **directional derivative** of the logit for class Doctor (h_{Doctor}) along the CAV direction:

$$S_{\text{Man,Doctor},l}(x) = \nabla_{a_l} h_{\text{Doctor}}(a_l(x)) \cdot \mathbf{v}^l_{\text{Man}} \tag{16}$$

A positive value ($S > 0$) indicates that moving activations in the direction of the concept *Man* directly increases the probability of classifying the person as a *Doctor*.

Step 4. Compute the TCAV Score:

- Calculate the proportion of Doctor images for which the concept has a positive causal influence:

$$\text{TCAV}_{\text{Man,Doctor},l} = \frac{|\{x \in X_{\text{Doctor}} : S_{\text{Man,Doctor},l}(x) > 0\}|}{|X_{\text{Doctor}}|} \in [0, 1] \tag{17}$$

Similarly compute $\text{TCAV}_{\text{Woman,Doctor},l}$.

Step 5. Statistical Significance Testing:

- To ensure that concepts are meaningful rather than noise artifacts, train multiple CAVs using different random negative sets (e.g., 50 random splits).
- Perform a two-sided t-test comparing the concept CAV scores against the distribution of random-versus-random CAV scores. Only concepts with $p < 0.01$ are considered statistically significant.

2. How TCAV Reveals Model Bias

TCAV provides an auditable diagnostic tool to unmask societal biases:

- **Quantitative Comparison:** Auditors evaluate $\text{TCAV}_{\text{Man,Doctor},l}$ against $\text{TCAV}_{\text{Woman,Doctor},l}$.

- **Diagnostic Signature of Bias:** Suppose the evaluation yields:

$$\text{TCAV}_{\text{Man,Doctor},l} = 0.88 \quad (p < 0.001) \quad \text{versus} \quad \text{TCAV}_{\text{Woman,Doctor},l} = 0.12 \quad (p > 0.05) \tag{18}$$

  This proves that for 88% of doctor test inputs, nudging the latent activations toward masculine traits increases the model's confidence in predicting "Doctor", while feminine traits either suppress or have null effect on the prediction.

- **Unmasking Spurious Correlations:** A medical model should base doctor classifications on clinical indicators: $\text{TCAV}_{\text{Stethoscope}} \approx 0.95$, $\text{TCAV}_{\text{WhiteCoat}} \approx 0.90$. If gender concepts yield high TCAV scores, it proves that the model's internal representations have learned societal gender stereotypes from historical training distributions.

- **Debiasing Actionability:** Pinpointing the exact layer l where gender bias emerges guides targeted interventions, such as concept scrubbing, projection onto the null-space of gender CAVs, or balanced data augmentation.

★ Examiner Feedback & Common Pitfalls (Prof. Eliana Pastor)

In official grading, students were docked points (−0.75 points total) for:

1. **Omitting the Negative / Random Datasets (−0.25 points):** A linear CAV cannot be learned with positive concept images alone; it requires random negative counter-examples to define the separating hyperplane.
2. **Failing to describe Directional Derivatives (−0.25 points):** Stating that TCAV measures correlation is insufficient; candidates must specify that it computes the directional derivative of the class logit along the CAV vector.
3. **Vague Bias Detection Description (−0.25 points):** Simply saying "it reveals bias" without explaining the comparative TCAV score mechanism (TCAV_{Man} vs. $\text{TCAV}_{\text{Woman}}$) and testing against clinically relevant baseline concepts.

➥ Course Theory Mapping (Notes & Slides)

NOTES/XAI_Theory.pdf – Chapter 7: "Concept-based Explainable AI", Section: "Testing with Concept Activation Vectors (T-CAV): Overview, Components, CAV Definition, Directional Derivatives, and TCAV Scores" (pp. 92–98).

¶ Alternative Perspectives & Methodological Variants

Relative TCAV trains a single binary linear classifier directly separating activations of *Man* from *Woman*, measuring whether the model systematically favors one demographic group over another along the continuous demographic axis.

Question 7: Origins of Noise in Vanilla Gradient Explanations MCQ 1.00 Point

Vanilla gradient explanations can be noisy because:

a. They average feature importance over the whole dataset

b. They require replacing features with unrealistic values

c. Small input changes can cause large gradient fluctuations

d. They only explain global model behavior, not local predictions

✓ Executive Solution Summary

The correct answer is **c**. Vanilla gradient explanations compute the local partial derivative $\nabla_x S_c(x)$. In deep non-linear neural networks (with millions of piecewise-linear ReLU activations and max-pooling operations), the loss landscape is locally jagged, fractured, and noisy. Consequently, **infinitesimal, visually imperceptible shifts in input pixels cause massive fluctuations in local gradient vectors**.

➤ Systematic Analysis of Multiple Choice Options

(c) [CORRECT] Small input changes can cause large gradient fluctuations

Deep neural networks are highly non-linear compositions. Even though the overall decision surface is smooth globally, the local derivative fluctuates rapidly at microscopic scales. This local instability manifests visually as high-frequency noise in gradient saliency maps.

(a) [INCORRECT] They average feature importance over the whole dataset

Vanilla gradients are strictly local to a single input instance x; they never average across the dataset (which describes global techniques like PFI).

(b) [INCORRECT] They require replacing features with unrealistic values

Gradients do not replace or permute features; they evaluate derivatives at the exact input coordinates x.

(d) [INCORRECT] They only explain global model behavior, not local predictions

Vanilla gradients explain individual local predictions, not global behavior.

✍ Comprehensive Technical Walkthrough

Let $S_c(x)$ be the class score. Vanilla gradient saliency defines attribution as:

$$A(x) = \nabla_x S_c(x) = \frac{\partial S_c(x)}{\partial x} \tag{19}$$

In deep networks with ReLU activations ($\sigma(z) = \max(0, z)$), the network is a piecewise linear function partitioned into an astronomical number of linear regions. As the input traverses the boundaries between these microscopic linear regions, activation patterns switch abruptly:

$$\lim_{\epsilon \to 0} \|\nabla_x S_c(x + \epsilon) - \nabla_x S_c(x)\| \gg 0 \tag{20}$$

This local gradient noise obscures the macroscopic semantic shape of the object.

The SmoothGrad Solution

Smilkov et al. (2017) addressed this exact gradient noise failure by introducing **SmoothGrad**, which smooths the local loss landscape by averaging gradient attributions across n Gaussian-perturbed samples:

$$A_{\text{SmoothGrad}}(x) = \frac{1}{n} \sum_{i=1}^{n} \nabla_x S_c \left(x + \mathcal{N}(0, \sigma^2 I) \right) \tag{21}$$

Averaging cancels out high-frequency gradient noise while preserving coherent macroscopic feature signals.

★ Examiner Feedback & Common Pitfalls (Prof. Eliana Pastor)

Do not confuse gradient fluctuations (a local derivative instability) with perturbation off-manifold sampling (which occurs in Permutation Feature Importance or Partial Dependence Plots).

➥ Course Theory Mapping (Notes & Slides)

NOTES/XAI_Theory.pdf – Chapter 5: "Gradient-based Explainability Methods", Sections: "Vanilla Gradient: Limitations" & "SmoothGrad" (pp. 68–71).

¶ Alternative Perspectives & Methodological Variants

Integrated Gradients (Sundararajan et al.) solves gradient noise and saturation simultaneously by integrating gradients along a straight line path from a neutral baseline x_0 to input x:

$$A_i^{\mathrm{IG}}(x) = (x_i - x_{0,i}) \times \int_0^1 \frac{\partial F(x_0 + \alpha(x - x_0))}{\partial x_i}\, d\alpha \tag{22}$$